Lab 7: Fuzz Testing for Input Validation Bugs

Hypothesis fuzzing of `income_tax_parser.py` (Income Tax Calculator)

Name	Catherine D
SRN	PES1UG24CS120
Section	5B
Assigned repo	SETAPESU26/L6_L7_57_income_tax (LAB7)
Submission repo	Catherine61005/SE-Labs-PES1UG24CS120 → `Lab-7/`
Date	11 November 2026

Contents

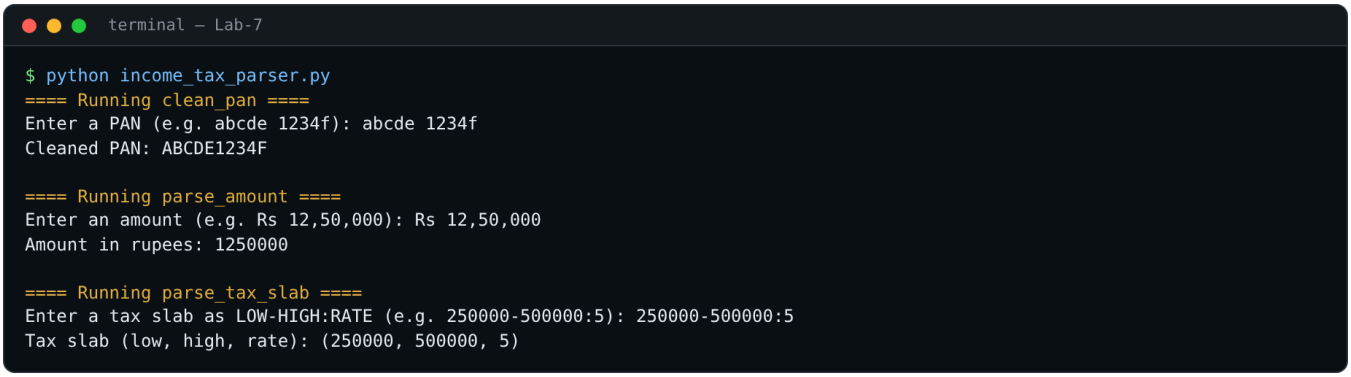
1. Execution of the original application (SS1) · 2. Failing fuzz tests (SS2)
3. Bugs found and fixes · 4. Execution of the corrected application (SS3)
5. Final test run (SS4) · 6. Source code: `income_tax_parser.py` and `test_income_tax_parser.py`

The handout calls the files `processor.py` / `test_processor.py`; in this assigned repo they are `income_tax_parser.py` / `test_income_tax_parser.py`, as stated in the LAB7 README.
Environment: Python 3.13.18, Hypothesis 6.170.15, pytest 9.2.0.

1. Execution of the original application (SS1)

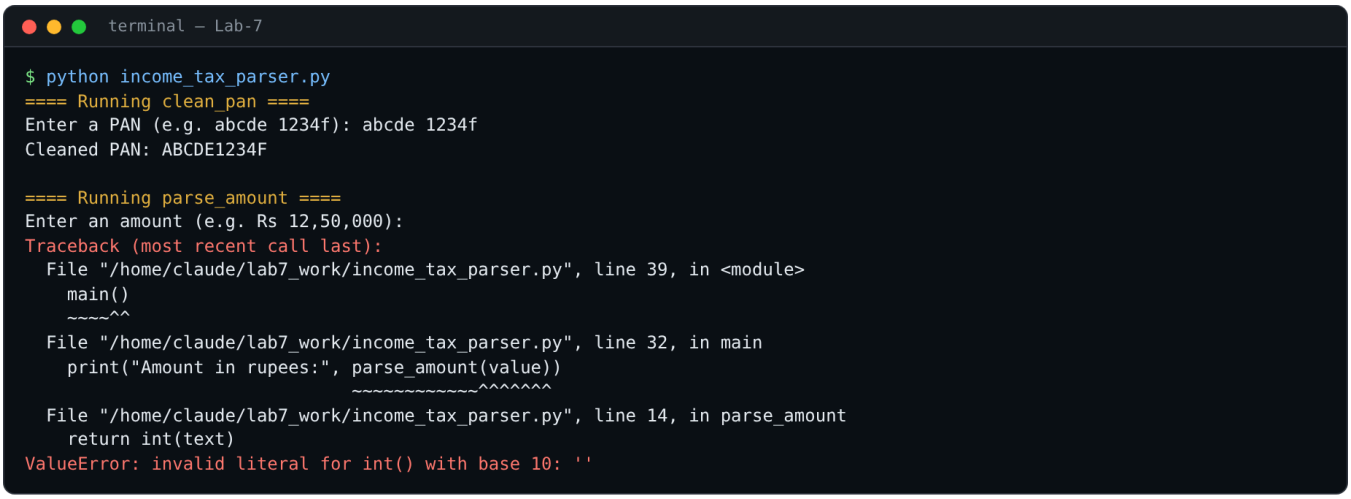
Step 2 of the handout: `python income_tax_parser.py` on the buggy module. With the well-formed example inputs from the prompts it works:

> Screenshots in this report show the exact output of each command, rendered in a terminal-window style from real runs.



SS1. Original application with well-formed input: it works.

The module assumes every input is well formed. A blank amount or a slab in another format makes it crash with a traceback:



SS1 (extra). Original application, empty amount: ValueError from int().



SS1 (extra). Original application, slab typed as ‘250000 to 500000’: ValueError while unpacking.

2. Failing test cases found by fuzzing (SS2)

Steps 3–4: `test_income_tax_parser.py` was completed with Hypothesis tests for all three functions (source in section 6). Running `pytest test_income_tax_parser.py` against the **buggy** module gives **17 failed, 2 passed**. The full output is 723 lines, so it is shown below in four contiguous excerpts: the start (session header, progress line, first failures), two of the multi-failure reports, and the end (last failure and the short test summary).

terminal — Lab-7

```
$ pytest test_income_tax_parser.py
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/claude/lab7_buggy
plugins: hypothesis-6.170.15
collected 19 items

test_income_tax_parser.py FFFF.FFFFFFFFF.FFF                             [100%]

=================================== FAILURES ===================================
_____________________________ test_clean_pan_no_crash _____________________________

s = None

    @given(st.text() | st.none())
    def test_clean_pan_no_crash(s):
        try:
>           clean_pan(s)

test_income_tax_parser.py:30:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

pan = None

    def clean_pan(pan):
        """
        Removes spaces from a PAN and upper-cases it.
        Assumes pan is a non-empty string.
        """
>       return pan.strip().replace(' ', '').upper()
               ^^^^^^^^^
E       AttributeError: 'NoneType' object has no attribute 'strip'

income_tax_parser.py:6: AttributeError

During handling of the above exception, another exception occurred:

    @given(st.text() | st.none())
>   def test_clean_pan_no_crash(s):
                ^^^

test_income_tax_parser.py:28:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = None

    @given(st.text() | st.none())
    def test_clean_pan_no_crash(s):
        try:
            clean_pan(s)
        except Exception as e:
>           assert False, f"clean_pan crashed with: {e!r}"
E           AssertionError: clean_pan crashed with: AttributeError("'NoneType' object has no attribute 'strip'")
E           assert False
E           Failing test case: test_clean_pan_no_crash(
E               s=None,
E           )
E           Explanation:
E               These lines were always and only run by failing test cases:
E                   /home/claude/lab7_buggy/test_income_tax_parser.py:31

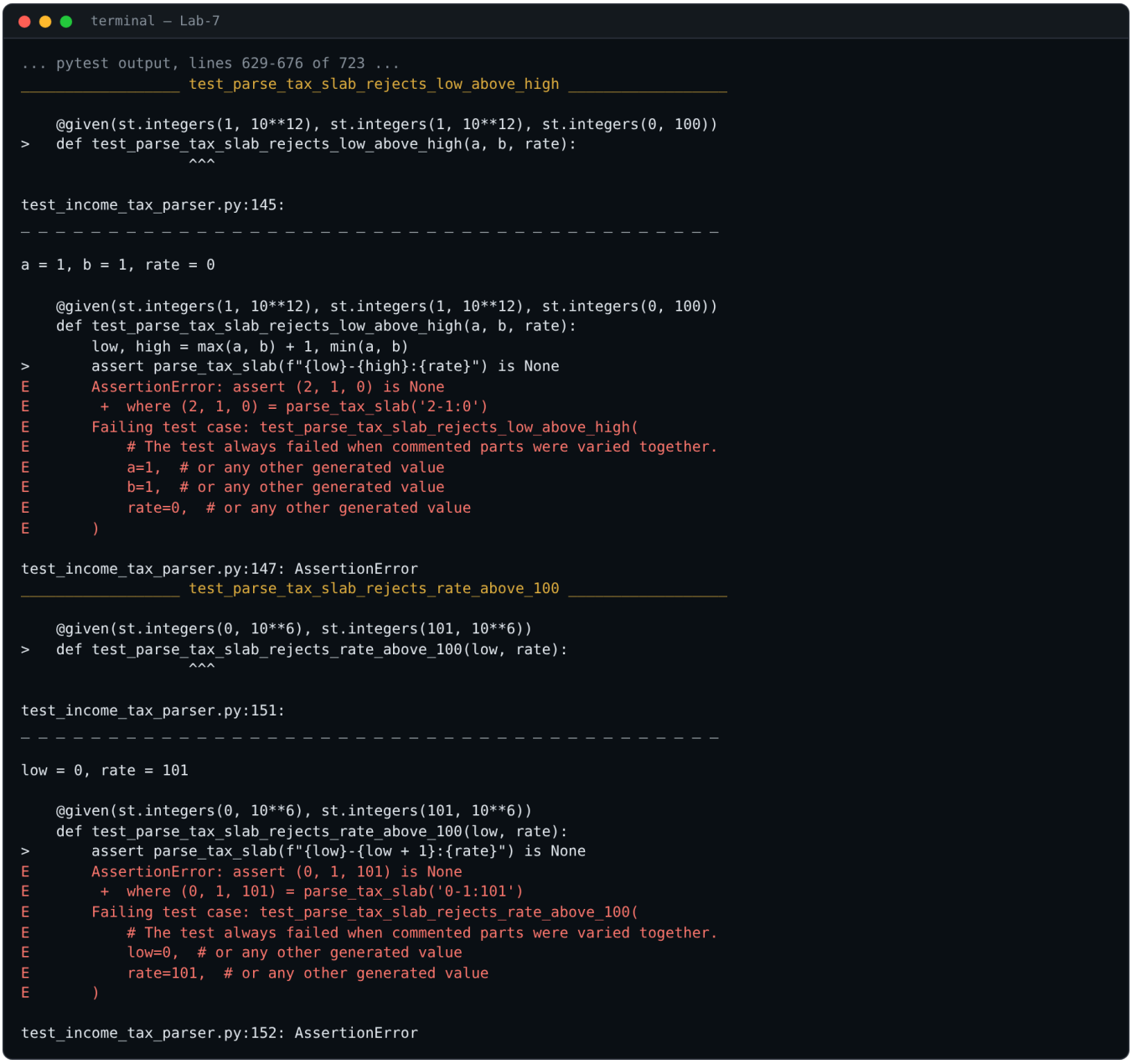
test_income_tax_parser.py:32: AssertionError
```

SS2 (1/4). Start of the run: 19 tests collected, `FFFF.FFFFFFFFF.FFF`. clean_pan crashes on `None` (AttributeError).

terminal — Lab-7

```
... pytest output, lines 166-217 of 723 ...
____________________________ test_parse_amount_no_crash ____________________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/claude/lab7_buggy/test_income_tax_parser.py", line 65, in test_parse_amount_no_crash
  |     def test_parse_amount_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/lab7_buggy/test_income_tax_parser.py", line 67, in test_parse_amount_no_crash
    |     parse_amount(s)
    |     ~~~~~~~~~~~~^^^
    |   File "/home/claude/lab7_buggy/income_tax_parser.py", line 14, in parse_amount
    |     return int(text)
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/claude/lab7_buggy/test_income_tax_parser.py", line 69, in test_parse_amount_no_crash
    |     assert False, f"parse_amount crashed with: {e!r}"
    | AssertionError: parse_amount crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_parse_amount_no_crash(
    |     s='',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /home/claude/lab7_buggy/test_income_tax_parser.py:68
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/lab7_buggy/test_income_tax_parser.py", line 67, in test_parse_amount_no_crash
    |     parse_amount(s)
    |     ~~~~~~~~~~~~^^^
    |   File "/home/claude/lab7_buggy/income_tax_parser.py", line 13, in parse_amount
    |     text = text.replace('Rs', '').replace(',', '').strip()
    |            ^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'replace'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/claude/lab7_buggy/test_income_tax_parser.py", line 69, in test_parse_amount_no_crash
    |     assert False, f"parse_amount crashed with: {e!r}"
    | AssertionError: parse_amount crashed with: AttributeError("'NoneType' object has no attribute 'replace'")
    | assert False
    | Failing test case: test_parse_amount_no_crash(
    |     s=None,
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /home/claude/lab7_buggy/test_income_tax_parser.py:68
    +-----------------------------------
```

SS2 (2/4). parse_amount: Hypothesis found 2 distinct failures — empty text and `None`.



SS2 (3/4). parse_tax_slab accepts a slab with LOW above HIGH, e.g. (2, 1, 0), and a rate above 100, e.g. (0, 1, 101).

terminal — Lab-7

```
... pytest output, lines 677-723 of 723 ...
_____________________ test_parse_tax_slab_rejects_line_without_colon _____________________

    @given(st.text().filter(lambda s: ":" not in s))
>   def test_parse_tax_slab_rejects_line_without_colon(s):
                   ^^^

test_income_tax_parser.py:156:

_ _ _ _
```

3. Bugs found and how they were fixed

Inputs in the second column are the (minimised) failing cases Hypothesis reported, plus a few related cases it implied.

Function	Bug in the original	Failing input	Fix
`clean_pan`	Crashes on non-text input	`None` → AttributeError	Return `""` if the input is not a string.
	Only plain spaces were removed; tabs and newlines stayed inside the PAN	`'0\n0'` returned as `'0\n0'`	Remove every whitespace character: `"".join(pan.split())`.
`parse_amount`	Crashes on non-text or empty/non-numeric text	`None`, `''`, `b''`, `'rs0'` → AttributeError / ValueError	Return `None` for non-strings; validate the text with a regex instead of calling `int()` blindly. 'Rs' is accepted in any case, also 'Rs.' and the rupee sign.
	Accepts negative amounts	`'-1'` → -1	Only ASCII digits are allowed, so a minus sign is rejected (`None`).
	Accepts other-script digits and underscores	`'٢'` (Arabic-Indic digit) → a number; `'1_000'` → 1000	Match `[0-9]` only (ASCII), no underscores.
	Crashes on a very long number (Python's 4300-digit `int()` limit, a denial-of-service risk)	`'9' * 4301` → ValueError	Reject numbers longer than 18 digits before converting (leading zeros are ignored).
	Crashes on decimal amounts	`'12.5'` → ValueError	Returns `None` (only whole rupees are accepted).
`parse_tax_slab`	Crashes on non-text, wrong separators or extra separators	`None`, `''`, `':'`, `'-:'`, `'1-2:5:6'`, `'-5-10:5'`	One strict pattern `LOW-HIGH:RATE` (ASCII digits, optional blanks); anything else returns `None`.
	Accepts LOW greater than HIGH	`'2-1:0'` → (2, 1, 0)	Return `None` if `low > high`.
	Accepts a tax rate above 100%	`'0-1:101'` → (0, 1, 101)	Return `None` if `rate > 100`.
	Accepts other-script digits and huge numbers	Arabic-Indic digits; 4301-digit limits	ASCII-only digits, at most 18 digits per limit.

Design decision: none of the three functions raises any more. Invalid input returns a safe value (`""` for `clean_pan`, `None` for the two parsers), and `main()` turns those into clear messages for the user (and handles Ctrl+C / closed input).

4. Execution of the corrected application (SS3)

Step 5: `python income_tax_parser.py` on the fixed module. Well-formed input gives the same results as before:

```
terminal — Lab-7

$ python income_tax_parser.py
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f): abcde 1234f
Cleaned PAN: ABCDE1234F

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): Rs 12,50,000
Amount in rupees: 1250000

==== Running parse_tax_slab ====
Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): 250000-500000:5
Tax slab (low, high, rate): (250000, 500000, 5)
```

SS3. Corrected application, well-formed input: unchanged results.

The inputs that used to crash (or be accepted wrongly) are now handled with a message:

```
terminal — Lab-7

$ python income_tax_parser.py
==== Running clean_pan ====
Enter a PAN (e.g. abcde 1234f):
Cleaned PAN: (nothing entered)

==== Running parse_amount ====
Enter an amount (e.g. Rs 12,50,000): 12.5
Invalid amount: use digits only, e.g. Rs 12,50,000

==== Running parse_tax_slab ====
Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): 250000 to 500000
Invalid tax slab: use LOW-HIGH:RATE with LOW <= HIGH and RATE 0-100, e.g. 250000-500000:5
```

SS3 (extra). Corrected application: empty PAN, a decimal amount and a wrongly formatted slab are handled without a crash.

5. Final test execution (SS4)

Step 6: `pytest test_income_tax_parser.py` against the fixed module — all 19 tests pass. The tests were also repeated with 10 different Hypothesis random seeds (300 examples per test each), and 900,000 random calls (text, None, numbers, bytes, lists) across the three functions raised no exception.



SS4. Final run: 19 passed.

6. Source code

6.1 `income_tax_parser.py` (fixed)

```python
"""Input parsing helpers for the income tax calculator (Lab 7, fixed version).

The original module assumed every input was well formed and crashed on
anything else (None, empty text, wrong format, huge numbers, ...).

Fixed behaviour: each function accepts *any* Python object and never raises.
  * clean_pan      -> always returns a str ("" if the input is not text)
  * parse_amount   -> returns a non-negative int, or None if the input is invalid
  * parse_tax_slab -> returns (low, high, rate) ints, or None if the input is invalid
"""
import re

MAX_DIGITS = 18   # largest amount/slab limit accepted (fits in 64 bits, avoids huge ints)
MAX_RATE = 100    # a tax rate is a percentage

# Optional 'Rs' / 'Rs.' / rupee-sign prefix (any case) followed by ASCII digits only.
_AMOUNT = re.compile(r"(?:rs\.?|₹)?([0-9]+)", re.IGNORECASE | re.ASCII)
# LOW-HIGH:RATE, ASCII digits only, optional blanks around the separators.
_SLAB = re.compile(
    r"\s*([0-9]{1,%d})\s*-\s*([0-9]{1,%d})\s*:\s*([0-9]{1,3})\s*" % (MAX_DIGITS, MAX_DIGITS)
)


def clean_pan(pan):
    """
    Removes all whitespace from a PAN and upper-cases it.
    Returns "" if pan is not a string (e.g. None), so it never raises.
    """
    if not isinstance(pan, str):
        return ""
    return "".join(pan.split()).upper()


def parse_amount(text):
    """
    Converts an amount such as 'Rs 12,50,000' into an int.
    Accepts an optional 'Rs' / 'Rs.' / rupee-sign prefix (any case), commas
    and spaces around plain ASCII digits.
    Returns None for anything else: non-text, empty text, decimals, negative
    numbers, other-script digits, underscores or numbers longer than MAX_DIGITS.
    """
    if not isinstance(text, str):
        return None
    match = _AMOUNT.fullmatch("".join(text.split()).replace(",", ""))
    if match is None:
        return None
    digits = match.group(1).lstrip("0") or "0"
    if len(digits) > MAX_DIGITS:
        return None
    return int(digits)


def parse_tax_slab(line):
    """
    Parses 'LOW-HIGH:RATE' into a (low, high, rate) tuple of ints.
    Returns None if the line is not in that exact format, if LOW is greater
    than HIGH, or if RATE is above MAX_RATE (100).
    """
    if not isinstance(line, str):
        return None
    match = _SLAB.fullmatch(line)
    if match is None:
        return None
    low, high, rate = (int(group) for group in match.groups())
    if low > high or rate > MAX_RATE:
        return None
    return low, high, rate


def main():
    try:
        print("==== Running clean_pan ====")
        value = input("Enter a PAN (e.g. abcde 1234f): ")
```

```
        pan = clean_pan(value)
        print("Cleaned PAN:", pan if pan else "(nothing entered)")

        print("\n==== Running parse_amount ====")
        value = input("Enter an amount (e.g. Rs 12,50,000): ")
        amount = parse_amount(value)
        if amount is None:
            print("Invalid amount: use digits only, e.g. Rs 12,50,000")
        else:
            print("Amount in rupees:", amount)

        print("\n==== Running parse_tax_slab ====")
        value = input("Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): ")
        slab = parse_tax_slab(value)
        if slab is None:
            print("Invalid tax slab: use LOW-HIGH:RATE with LOW <= HIGH and RATE 0-100,"
                  " e.g. 250000-500000:5")
        else:
            print("Tax slab (low, high, rate):", slab)
    except (EOFError, KeyboardInterrupt):
        print("\nInput cancelled.")


if __name__ == "__main__":
    main()
```

6.2 `test_income_tax_parser.py` (completed Hypothesis tests)

```
"""Hypothesis fuzz tests for income_tax_parser.py (Lab 7).

Each of the three functions gets:
  * a "no crash" fuzz test (the handout template: any text or None must not raise),
  * a wider "no crash" test fed with values of *any* type,
  * property tests that check the answer is correct, not just that it exists.
"""
import string

from hypothesis import given, settings, strategies as st

from income_tax_parser import clean_pan, parse_amount, parse_tax_slab

settings.register_profile("lab7", max_examples=300, deadline=None)
settings.load_profile("lab7")

# Anything a caller could hand to the functions: text, None and other types.
ANY_INPUT = st.one_of(
    st.text(), st.none(), st.integers(), st.floats(), st.booleans(),
    st.binary(), st.lists(st.text(), max_size=3),
)

MAX_AMOUNT_DIGITS = 18   # amounts longer than this are rejected by the fixed parser


# ---------------------------------------------------------------- clean_pan
@given(st.text() | st.none())
def test_clean_pan_no_crash(s):
    try:
        clean_pan(s)
    except Exception as e:
        assert False, f"clean_pan crashed with: {e!r}"


@given(ANY_INPUT)
def test_clean_pan_no_crash_any_type(s):
    try:
        clean_pan(s)
    except Exception as e:
        assert False, f"clean_pan crashed with: {e!r}"


@given(st.text() | st.none())
def test_clean_pan_output_is_clean(s):
    """Result is always a string, upper-case, with no whitespace left in it."""
    out = clean_pan(s)
    assert isinstance(out, str)
    assert out == out.upper()
    assert not any(ch.isspace() for ch in out)
```

```python


@given(st.text(alphabet=string.ascii_letters + string.digits + " \t\n"))
def test_clean_pan_removes_whitespace_and_uppercases(s):
    expected = "".join(ch for ch in s if not ch.isspace()).upper()
    assert clean_pan(s) == expected


@given(st.text())
def test_clean_pan_is_idempotent(s):
    assert clean_pan(clean_pan(s)) == clean_pan(s)


# ------------------------------------------------------------ parse_amount
@given(st.text() | st.none())
def test_parse_amount_no_crash(s):
    try:
        parse_amount(s)
    except Exception as e:
        assert False, f"parse_amount crashed with: {e!r}"


@given(ANY_INPUT)
def test_parse_amount_no_crash_any_type(s):
    try:
        parse_amount(s)
    except Exception as e:
        assert False, f"parse_amount crashed with: {e!r}"


@given(ANY_INPUT)
def test_parse_amount_result_is_none_or_non_negative_int(s):
    result = parse_amount(s)
    assert result is None or (isinstance(result, int) and result >= 0)


@given(st.integers(min_value=0, max_value=10**15))
def test_parse_amount_round_trip(n):
    assert parse_amount(str(n)) == n
    assert parse_amount(f"Rs {n:,}") == n
    assert parse_amount(f"rs{n:,}") == n


@given(st.integers(max_value=-1))
def test_parse_amount_rejects_negative(n):
    assert parse_amount(str(n)) is None


@given(st.text(alphabet=st.characters(categories=["Nd"], exclude_characters="0123456789"),
               min_size=1))
def test_parse_amount_rejects_non_ascii_digits(s):
    # e.g. Arabic-Indic digits: int() would accept them, an amount should not
    assert parse_amount(s) is None


@given(st.integers(min_value=MAX_AMOUNT_DIGITS + 1, max_value=6000))
def test_parse_amount_rejects_absurdly_long_numbers(n):
    # also covers Python's 4300-digit int() limit, which used to raise ValueError
    assert parse_amount("9" * n) is None


# ------------------------------------------------------------ parse_tax_slab
@given(st.text() | st.none())
def test_parse_tax_slab_no_crash(s):
    try:
        parse_tax_slab(s)
    except Exception as e:
        assert False, f"parse_tax_slab crashed with: {e!r}"


@given(ANY_INPUT)
def test_parse_tax_slab_no_crash_any_type(s):
    try:
        parse_tax_slab(s)
    except Exception as e:
        assert False, f"parse_tax_slab crashed with: {e!r}"


@given(ANY_INPUT)
def test_parse_tax_slab_result_is_none_or_valid_slab(s):
    result = parse_tax_slab(s)
```

```
    if result is not None:
        low, high, rate = result
        assert all(isinstance(v, int) for v in result)
        assert 0 <= low <= high
        assert 0 <= rate <= 100


@given(st.integers(0, 10**12), st.integers(0, 10**12), st.integers(0, 100))
def test_parse_tax_slab_round_trip(a, b, rate):
    low, high = min(a, b), max(a, b)
    assert parse_tax_slab(f"{low}-{high}:{rate}") == (low, high, rate)


@given(st.integers(1, 10**12), st.integers(1, 10**12), st.integers(0, 100))
def test_parse_tax_slab_rejects_low_above_high(a, b, rate):
    low, high = max(a, b) + 1, min(a, b)
    assert parse_tax_slab(f"{low}-{high}:{rate}") is None


@given(st.integers(0, 10**6), st.integers(101, 10**6))
def test_parse_tax_slab_rejects_rate_above_100(low, rate):
    assert parse_tax_slab(f"{low}-{low + 1}:{rate}") is None


@given(st.text().filter(lambda s: ":" not in s))
def test_parse_tax_slab_rejects_line_without_colon(s):
    assert parse_tax_slab(s) is None
```